EE 5239 Nonlinear Optimization

Lecture 0: Introduction

Mingyi Hong

University of Minnesota

Outline

Organization

Instructors:
Prof. Mingyi Hong (mhong@umn.edu)
6-109 Keller Hall
Dr. Jiaxiang Li (li003755@umn.edu)
5-151 Keller Hall

Administrative Details:
Lecture time: MonWed 13:00pm-14:15pm
Location: Keller 5-151
Office hour: 14:15 – 3:00pm (Mon), 16:00 - 17:00pm (Wed)
Office hour location: Keller 6-109 (Monday) Keller 5-151 (Wednesday)

TA: Bingqing Song (song0409@umn.edu)
Office hours: 11- noon Tue/Thur
Office hour location: Keller 2-276

Course Website: Canvas

Organization

Textbook

D. Bertsekas, *Nonlinear Programming* (V3 preferred), Main Textbook

J. Nocedal and S. Wright, *Numerical Optimization*, Reference

D. Bertsekas and J. Tsitsiklis, *Neurao-Dynamic Programming* Reference

Other relevant materials, and class notes will be distributed and discussed throughout the course

Syllabus

1. Can be found on the course website
2. Some of the key points here
3. Office hours
   - **Instructor Hong:** Mondays, 2:15 – 3pm
   - **Instructor Li:** Wednesdays, 4pm – 5pm
   - Or by appointment
4. The lecture slides will be uploaded one day ahead of time

Syllabus

1. Numerical grade = Homework (30%) + exam (35%) + class project (30%) + bonus points (up to 5%)
2. All homework will be submitted electronically, in pdf, through Canvas
3. Two homeworks have to be typed
4. After homework or exams are graded, one week to discuss with me/TA about the grades
5. Late homework and exam will not be accepted or graded
6. Bonus points = bonus problems and/or excellent project

Components of the course

1. Homework will be assigned regularly, some of them are about mathematical derivations, some of them requires programming
2. Course slides will be distributed regularly, but not all results/materials will be on the slides
3. Course notes will be distributed regularly as well
4. One mid-term (tentative Oct 30)
5. Class project report
6. No final

More on the course

1. Lectures will be delivered using both slides and discussion on board
2. The course is relatively self-contained, not too much requirements on previous courses on optimization
3. But basic knowledge about linear algebra is required
4. The course will often focus on theoretical analysis, but will try to provide insights/discussion/applications whenever possible
5. All three books are excellent references to have
6. Happy to take any suggestions/comments about the course/teaching during the semester

Project

1. There is a class project (30% of the numerical grade)
2. Two milestones
   - Form a team and submit a project proposal by Oct 3rd (tentative), through Canvas (1-2 persons per team)
   - Submit a project report by Dec 19th through Canvas; Graded by the instructor, worth 30% of the grade

Academic Misconduct

1. There is zero tolerance on academic misconduct. Individuals suspected of committing academic dishonesty will be directed to the Dean of Students Office as per University policy. Penalty for academic misconduct (up to 100%).

2. Collaboration is encouraged, but not copying each others' homeworks

Outline

What is Optimization

$$\begin{aligned} \text{minimize} \quad & f(x) \\ \text{subject to} \quad & x \in X \end{aligned} \tag{0.1}$$

- x: the decision variable (discrete, continuous)
- f : the objective function (differentiable, convex, linear...)
- X: the feasible region (convex, nonempty,...)

Key questions:

- When is the problem feasible?
- Does the optimal solution exist?
- How to determine if a feasible x is an optimal solution?
- How to find an optimal solution? (not by exhaustive search)

How To Use Optimization

Questions

1. Which algorithm should I use?
2. How fast should I get my results?
3. Is my problem easy to solve?
4. How do I know that the answer I got is global min?
5. My cost function is nondifferentiable, what should I do?
6. Why is my algorithm becomes very very slow?
7. What if I only can obtain input output of my problem, but not its explicit expression?

Answer: This course

The Main Theoretic Topics to be Covered

- Unconstrained Optimization
  1. Optimality Conditions
  2. Gradient/Newton's Method
  3. Conjugate Gradient Method
- Constrained Optimization
  1. Optimality Condition
  2. Descent Methods
  3. Conditional Gradient Method
  4. Block Coordinate Descent Method
- Lagrangian Multiplier Theory
  1. Equality Constrained Problems
  2. Inequality Constrained Problems
  3. Sensitivity Analysis
  4. Linearly Constrained Problems
  5. Duality Theory

The Main Theoretic Topics to be Covered

- Lagrangian Multiplier Method
  1. Penalty Method
  2. Method of Multipliers
- Issues of Large Scale Optimization
- Proximity, Bregmann Distance
- Accelerated PG Method, Incremental PG Method, Stochastic Optimization
- Applications in Signal Processing, Communication, Machine Learning, and others will be discussed together as the course continues

Outline

Example 1: Regression Problem

1. **Training data sets** (n data points, $\mathbf{a}_i$ is the "feature", and b_i is the "label")

$$\{\mathbf{a}_i, b_i\}_{i=1,\cdots,n},\ \mathbf{a}_i \in \mathbb{R}^d,\ b_i \in \mathbb{R}$$

2. **Objective**: Learn a predictive function, characterized by $\mathbf{x} \in \mathbb{R}^d, x_0 \in \mathbb{R}$,

$$f(\mathbf{a}; \mathbf{x}) = \mathbf{x}^T\mathbf{a} + x_0 = \tilde{\mathbf{x}}^T\tilde{\mathbf{a}}, \quad \text{with} \quad \tilde{\mathbf{x}} := [\mathbf{x}, x_0],\ \tilde{\mathbf{a}} := [\mathbf{a}, 1]$$

3. We have absorbed x_0 in $\mathbf{x}$ and augmenting $\mathbf{a}_i$'s with extra 1

Example 1: Regression Problem

1. **Training data sets** (n data points, $\mathbf{a}_i$ is the "feature", and b_i is the "label")

$$\{\mathbf{a}_i, b_i\}_{i=1,\cdots,n},\ \mathbf{a}_i \in \mathbb{R}^d,\ b_i \in \mathbb{R}$$

2. **Objective**: Learn a predictive function, characterized by $\mathbf{x} \in \mathbb{R}^d, x_0 \in \mathbb{R}$,

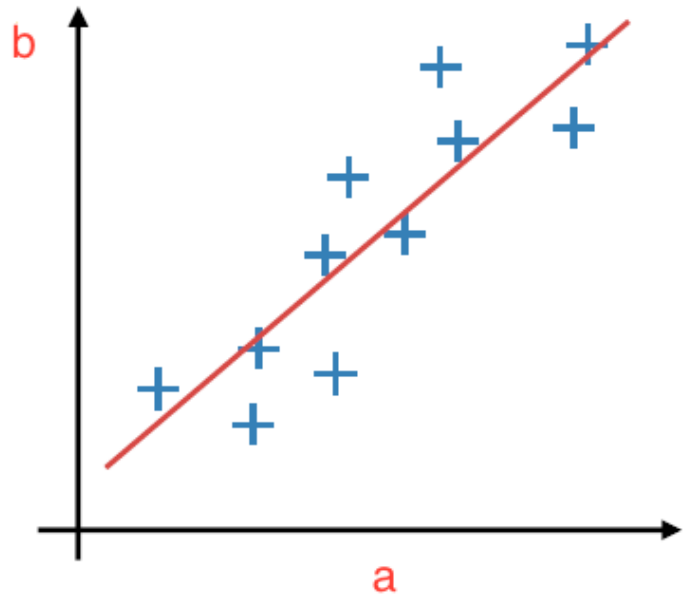
$$f(\mathbf{a}; \mathbf{x}) = \mathbf{x}^T\mathbf{a} + x_0 = \tilde{\mathbf{x}}^T\tilde{\mathbf{a}}, \quad \text{with} \quad \tilde{\mathbf{x}} := [\mathbf{x}, x_0],\ \tilde{\mathbf{a}} := [\mathbf{a}, 1]$$

3. We have absorbed x_0 in $\mathbf{x}$ and augmenting $\mathbf{a}_i$'s with extra 1
4. Use the cost function

$$\text{Loss}(\mathbf{x}) = \underbrace{\frac{1}{2}\sum_{i=1}^{n}(\tilde{\mathbf{x}}^T\tilde{\mathbf{a}}_i - b_i)^2}_{\text{squared loss}} = \|\mathbf{A}^T\tilde{\mathbf{x}} - \mathbf{b}\|^2$$

Minimize w.r.t. $\tilde{\mathbf{x}}$: $\tilde{\mathbf{x}}^* = \left(\mathbf{A}\mathbf{A}^T\right)^{-1}\mathbf{A}\mathbf{b}$ (closed-form solution!)

Example 1: Regression Problem



Example 1: Regression Problem

- It is the basic regression model; the predicted quantities are numerical
- Many aspects (including modeling and algorithm) can be much improved, and required a lot thoughtful refinements, for example when:

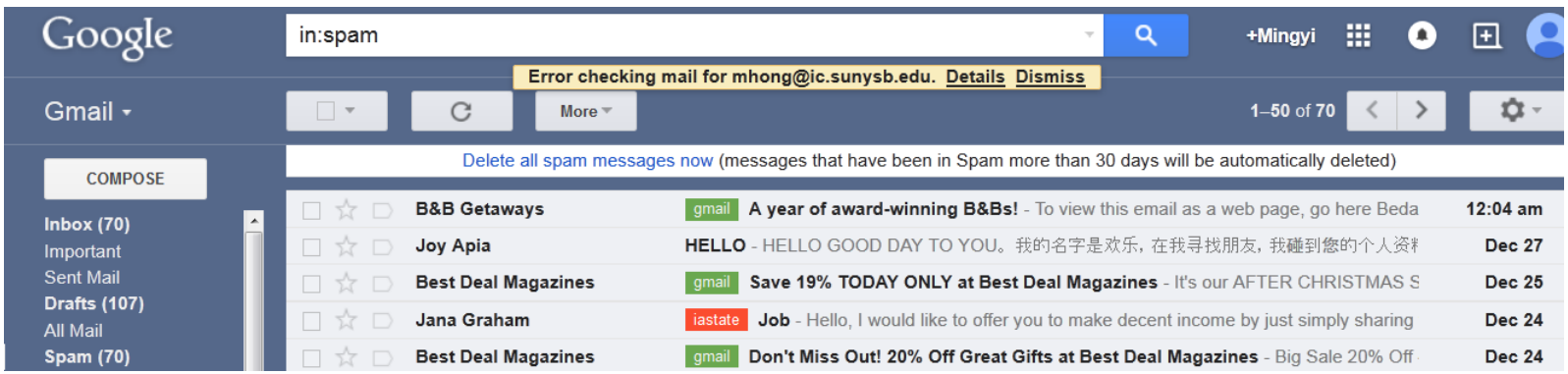
  1. The data matrix $\mathbf{A}$ becomes "large"
  2. The matrix $\left(\mathbf{A}\mathbf{A}^T\right)$ is not invertible
  3. The dimension of the optimization problem $\mathbf{x}$ becomes large
  4. There are special structures needed in $\mathbf{x}$

Example 1: Regression Problem

- What if we also want to select a few key features that are most important?
- What if the data sets are arriving sequentially?
- What if the data sets are distributed at different locations?
- What if the dimension of the variable is huge ($\mathbf{x}$ very long, lots of features), while the data is scarce (n is small)?
- What if the relationship is nonlinear?

Example 2: Classification

- Every email services have automatic spam detection mechanisms
- The basic task is the following
  1. Given an excerpt of an email, represented by a vector $\mathbf{a}_i$, where the elements can be words from the email
  2. Ask the question whether it is a spam or not ($b_i = +1, -1$)

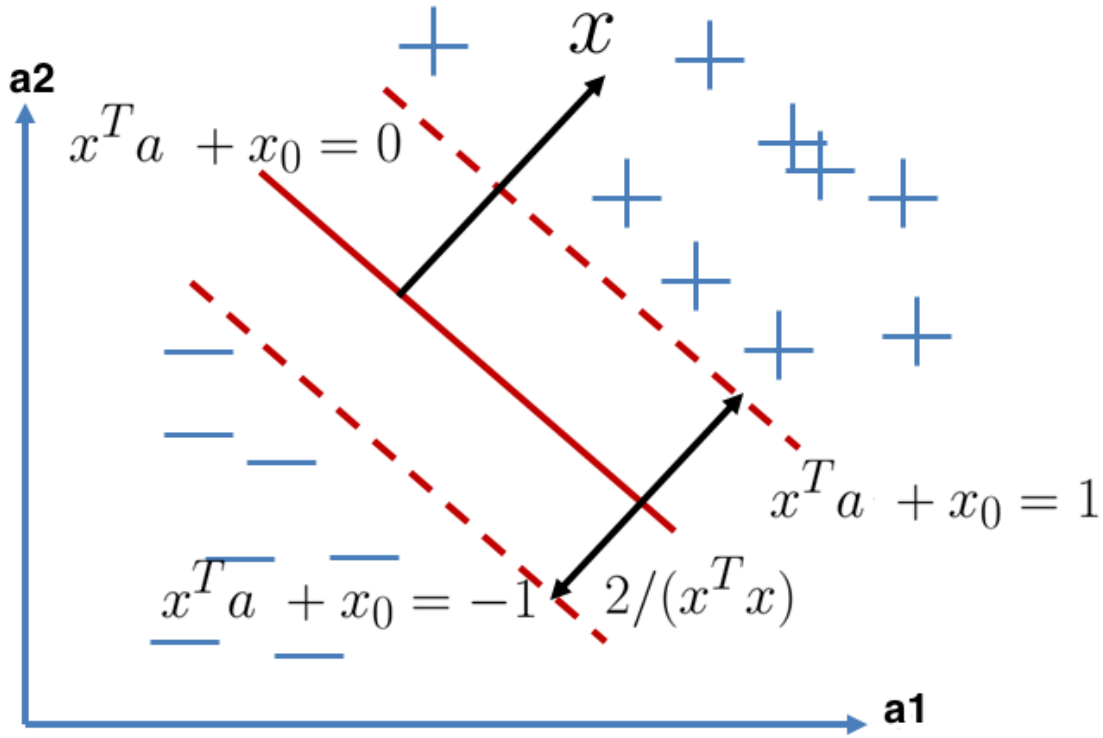


Example 2: Classification (cont.)

- **Training Stage**: data set $\{\mathbf{a}_i, b_i\}_{i=1}^n$, $\mathbf{a}_i \in \mathbb{R}^d$, $b_i \in \{-1, +1\}$
- $b_i = +1$ (spam); $b_i = -1$ (regular)
- **Objective**: learn the classification function
  $f(\mathbf{a}; \mathbf{x}) = \text{sgn}(\mathbf{x}^T\mathbf{a} + x_0)$, where sgn is the "sign function"
- **Testing stage**
  1. Given a feature vector $\widehat{\mathbf{a}}$
  2. Plug in $f(\mathbf{a}; \mathbf{x})$
  3. Classify based on the sign (i.e., "+" as spam, "-" as regular)
- Suppose the training data are linearly separable (?)
- **Task 1**: Select two hyperplanes to separate the data points
- **Task 2**: Try to maximize their distance
- What's a good formulation?

Example 2: Classification (cont.)

- **Intuition**: Find the separating plane that is far away from both classes



Example 2: Classification (cont.)

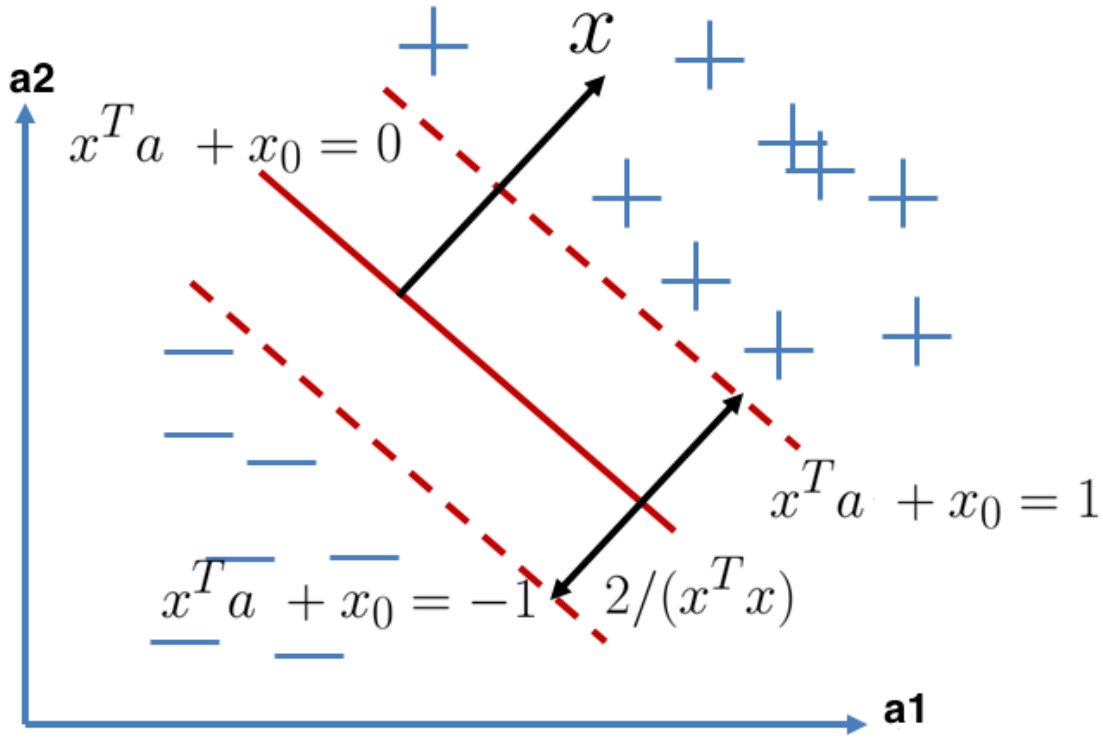
- **Formulation**: Consider the following problem

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{x}^T\mathbf{x} \\ \text{subject to} \quad & b_i(\mathbf{x}^T\mathbf{a}_i + x_0) \geq 1, \ \forall \ i \end{aligned}$$

- One data point, one constraint
- The blue part says there is no classification error
- If $b_i = 1$ (is a spam), then $(\mathbf{x}^T\mathbf{a}_i + x_0) \geq 1$
- If $b_i = -1$ (regular), then $(\mathbf{x}^T\mathbf{a}_i + x_0) \leq 1$

Example 2: Classification (cont.)

- **Intuition**: Find the separating plane that is far away from both classes



Example 2: Classification (cont.)

- This is one instance of the support vector machine (SVM) problem
- How about non-separable cases?
- How to solve the underlying optimization problem?
- Directly or need some re-formulation?
- What if the decision boundary is nonlinear?
- Data comes in sequentially?
- ....

Example 3: Training Neural Networks

- Neural Networks, especially Deep Neural Networks (DNN) become increasingly popular for various machine learning tasks
- Lots of high-profile applications: "Google Brain" by Google, 2012
- $16,000$ computer processors and 100 million images, $20,000$ distinct items, look for cats; research paper can be found here
- A nice article about the history of DL and NN on here

Example 3: Training Neural Network

Finding nonlinear classification boundary?

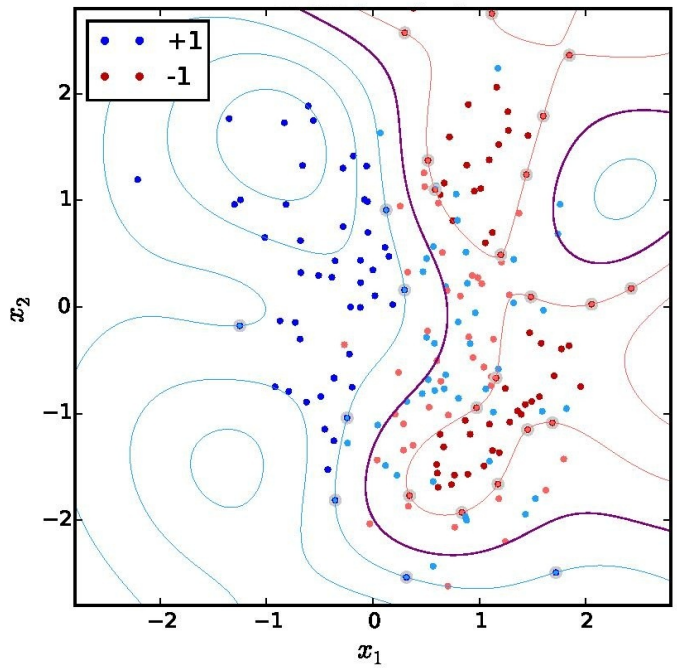
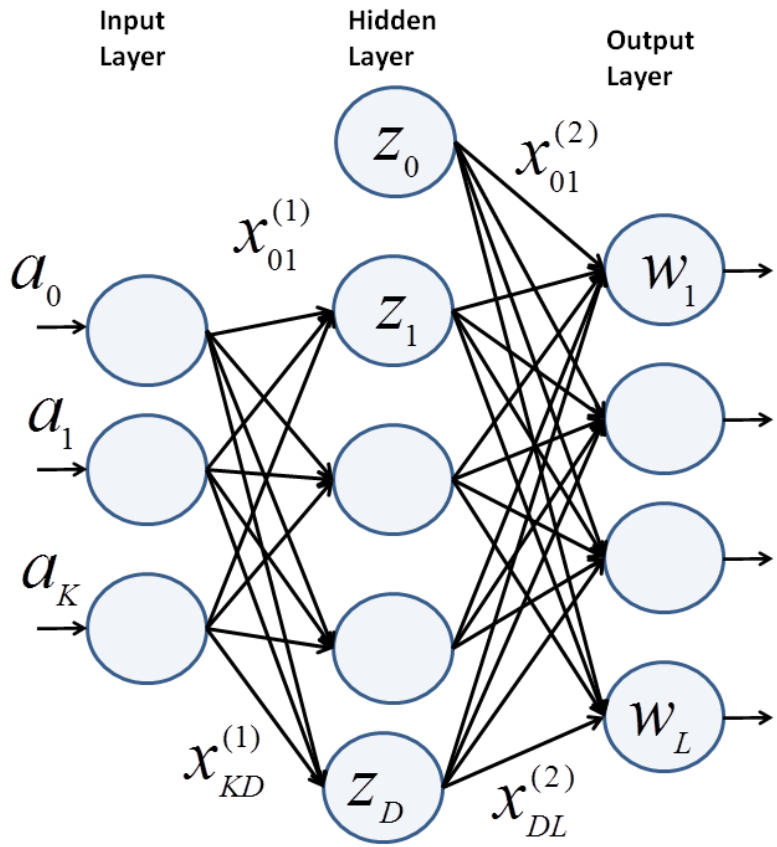


Figure 0.1: Nonlinear Classification [Wikipedia]

Example 3: Training Neural Network

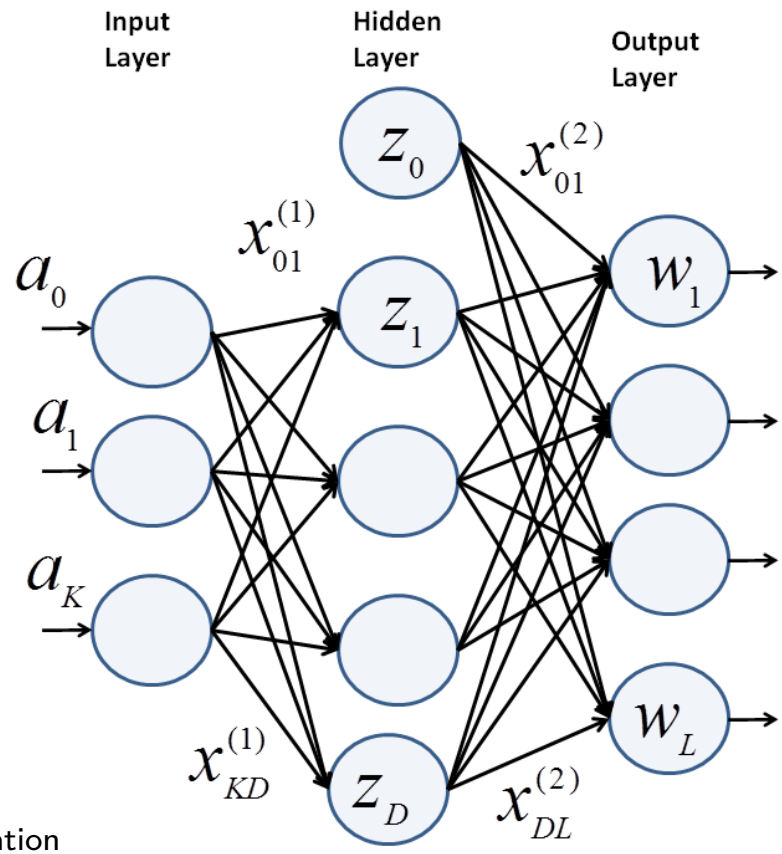
- **Input layer**: Each input one data point, K-dimensional: $a_0, \cdots, a_k$
- Note: Here we have a single data point, a_k denotes its kth element



Example 3: Training Neural Network

- **Hidden layer**: Each node, a nonlinear function g (tanh, sigmoid)
- Nonlinearly transform the inputs to outputs

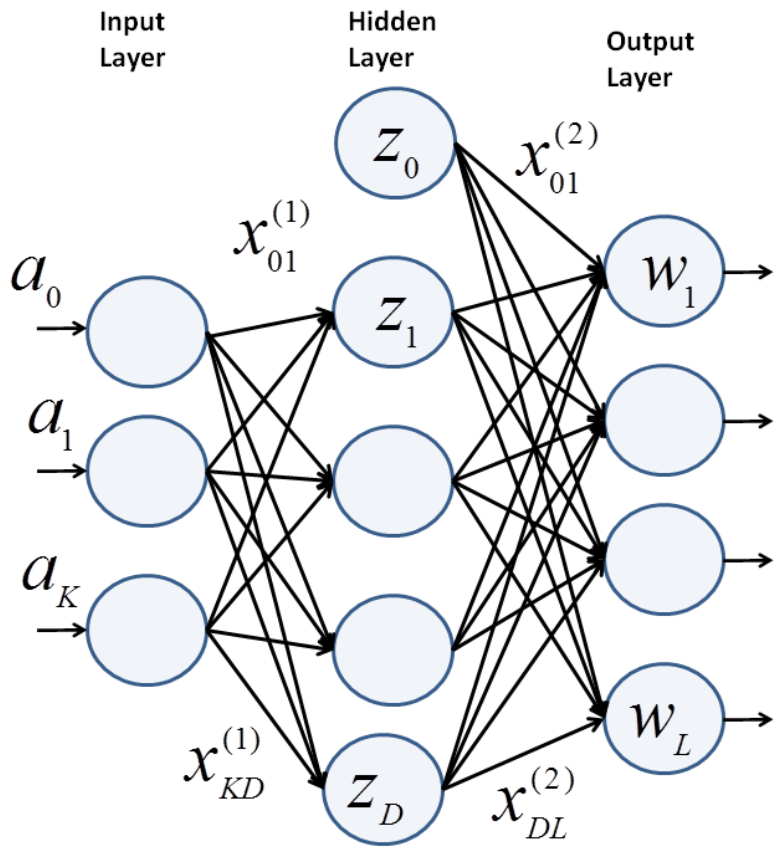
$$z_d = g\left(\sum_{k=1}^{K} a_k x_{k,d}^{(1)}\right)$$



Example 3: Training Neural Network

- **Output layer**: Each output node, a nonlinear function (sigmoid)

$$w_\ell = h\left(\sum_{d=0}^{D} z_d x_{d,\ell}^{(2)}\right), \quad \text{if binary classification, } L = 1$$



Example 3: Training Neural Network

- What are the optimization variables here?

Example 3: Training Neural Network

- What are the optimization variables here?
- How to setup the optimization problem?

$$\min_{\mathbf{x}} \sum_i \|g(\mathbf{x}; \mathbf{a}_i) - b_i\|^2 \tag{0.1}$$

where $g(\mathbf{x}; \mathbf{a}_i)$ indicates the output of the neural network, parameterized by $\mathbf{x}$, when the input is $\mathbf{a}_i$

Example 3: Training Neural Network

- What are the optimization variables here?
- How to setup the optimization problem?

$$\min_{\mathbf{x}} \sum_i \|g(\mathbf{x}; \mathbf{a}_i) - b_i\|^2 \tag{0.1}$$

  where $g(\mathbf{x}; \mathbf{a}_i)$ indicates the output of the neural network, parameterized by $\mathbf{x}$, when the input is $\mathbf{a}_i$

- How to solve the resulting problem?

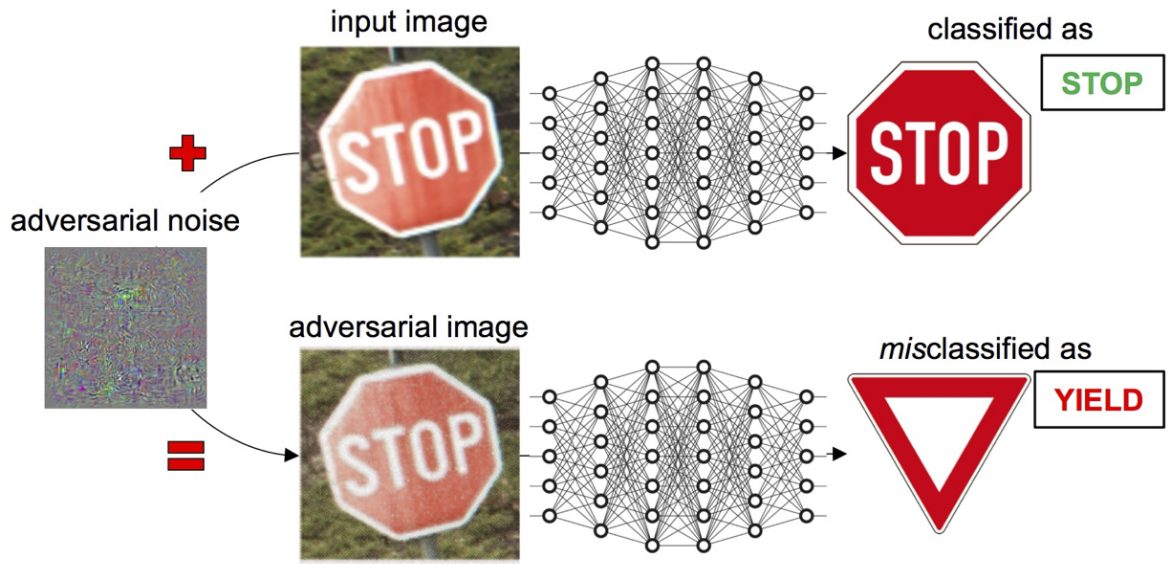
Example 4: Adversary Learning

Deep learning (DL) has been very successful in recent years

- Image/voice recognition
- Video analysis
- Machine translation
- Etc

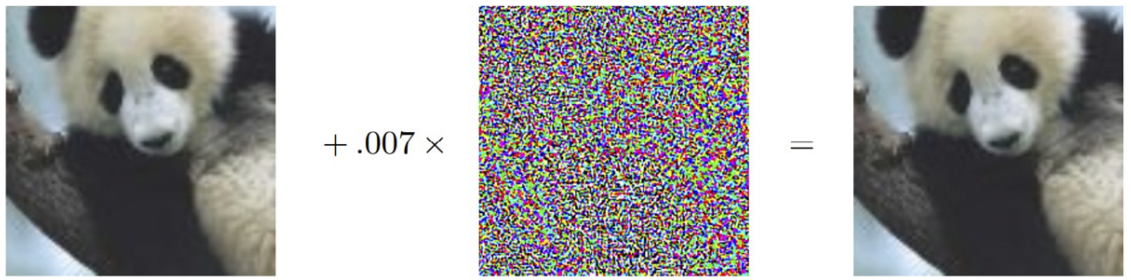
Example 4: Adversary Learning

- It could be catastrophic if ML models can be easily and intentionally tempered with



Example 4: Adversary Learning

- Indeed, there are examples that are intentionally crafted to confuse ML models



Example 4: Adversary Learning

- It turns out that, by properly setting up an optimization problem, and apply algorithms to be learned in this class, it is very easy to generate these adversary examples

Example 4: Adversary Learning

- It turns out that, by properly setting up an optimization problem, and apply algorithms to be learned in this class, it is very easy to generate these adversary examples
- A simple and effective attack [Goodfellow et al 14]

Example 4: Adversary Learning

- It turns out that, by properly setting up an optimization problem, and apply algorithms to be learned in this class, it is very easy to generate these adversary examples
- A simple and effective attack [Goodfellow et al 14]
- Given image a, label b, model parameter x and loss $L(x; a, b)$
- If we are to find the parameter (neural network model) that best fit the data, we should do:

$$\min_x \; L(x; a, b) \tag{0.2}$$

Example 4: Adversary Learning

- It turns out that, by properly setting up an optimization problem, and apply algorithms to be learned in this class, it is very easy to generate these adversary examples
- A simple and effective attack [Goodfellow et al 14]
- Given image a, label b, model parameter x and loss $L(x; a, b)$
- If we are to find the parameter (neural network model) that best fit the data, we should do:

$$\min_x \; L(x; a, b) \tag{0.2}$$

- Now I want to find a perturbation of the input image, such that the output get messed up; what should I do?

Example 5: Adversary Learning

- **First try:** Since we want to mess up the output, we might consider the following optimization problem

$$\max_{\delta} L(x; a + \delta, b) \tag{0.3}$$

Example 5: Adversary Learning

- **First try:** Since we want to mess up the output, we might consider the following optimization problem

$$\max_{\delta} L(x; a + \delta, b) \tag{0.3}$$

- Here, δ becomes the optimization variable; It is the noise to be added to the input data; We are finding the noise vector to be put on the pixels, so that we are make the loss as large as possible

Example 5: Adversary Learning

- **First try:** Since we want to mess up the output, we might consider the following optimization problem

$$\max_{\delta} L(x; a + \delta, b) \tag{0.3}$$

- Here, δ becomes the optimization variable; It is the noise to be added to the input data; We are finding the noise vector to be put on the pixels, so that we are make the loss as large as possible
- Are we done yet?

Example 5: Adversary Learning

- **First try:** Since we want to mess up the output, we might consider the following optimization problem

$$\max_{\delta} L(x; a + \delta, b) \tag{0.3}$$

- Here, δ becomes the optimization variable; It is the noise to be added to the input data; We are finding the noise vector to be put on the pixels, so that we are make the loss as large as possible
- Are we done yet?
- If we do not put any constraint on the noise, what will happen?

Example 5: Adversary Learning

- **First try:** Since we want to mess up the output, we might consider the following optimization problem

$$\max_{\delta} \; L(x; a + \delta, b) \tag{0.3}$$

- Here, δ becomes the optimization variable; It is the noise to be added to the input data; We are finding the noise vector to be put on the pixels, so that we are make the loss as large as possible
- Are we done yet?
- If we do not put any constraint on the noise, what will happen?
- A modified formulation

$$\max_{\delta} \quad L(x; a + \delta, b) \tag{0.4}$$

$$\text{s.t.} \quad \|\delta\| \leq \epsilon \tag{0.5}$$

Example 5: Adversary Learning

- What is a good algorithm to solve the problem?

Example 5: Adversary Learning

- What is a good algorithm to solve the problem?
- Now we only mess up the output (but without knowing what the model will predict); how do we make the model change a stop sign to a specific target (say yield)?

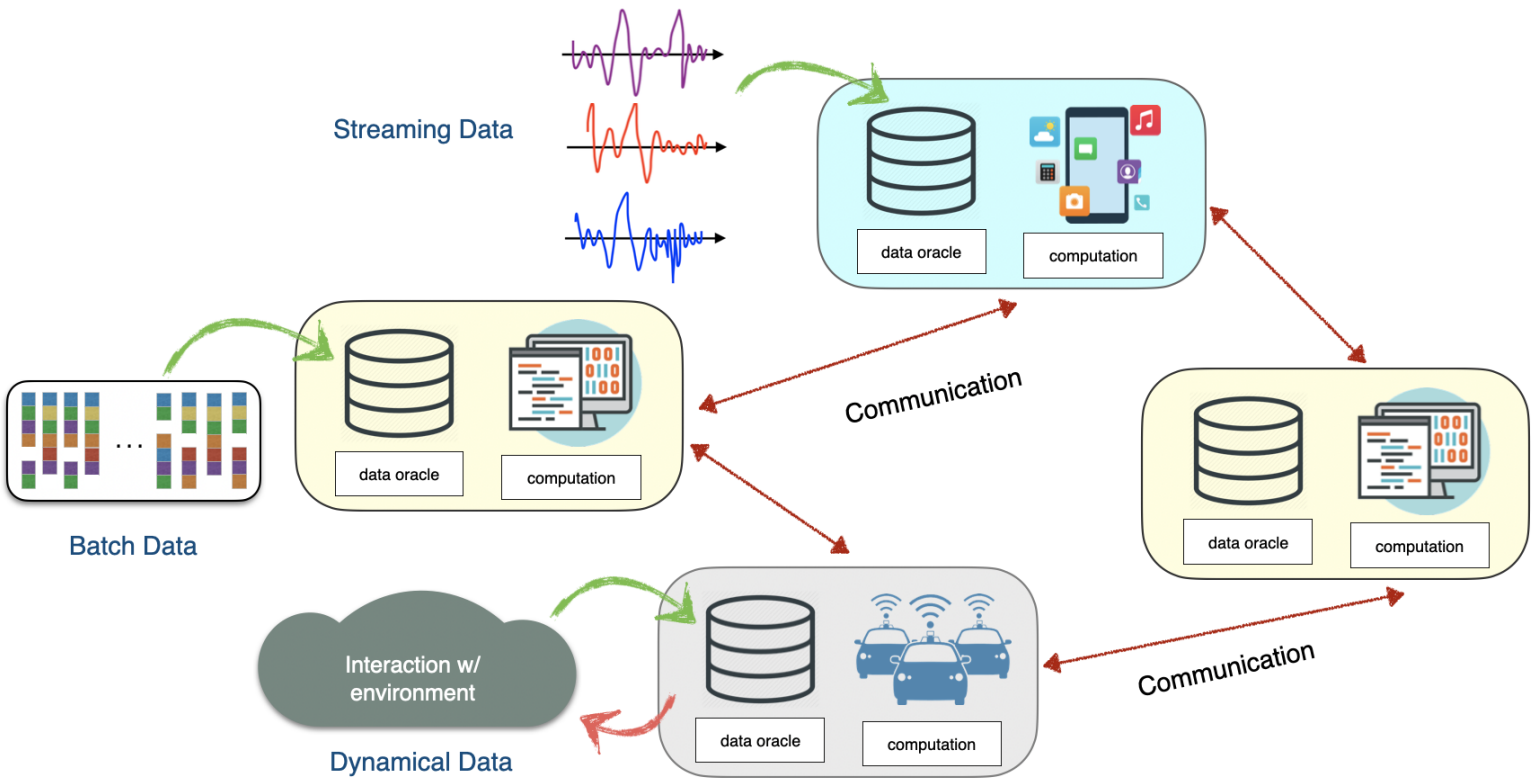
Example 5: Adversary Learning

- What is a good algorithm to solve the problem?
- Now we only mess up the output (but without knowing what the model will predict); how do we make the model change a stop sign to a specific target (say yield)?
- What if we do not have direct access of the model (think about you are interacting with a cloud service, do you can only inquire and get feedback, but do not know what the models is, so you cannot directly optimize)

Example 6: Decentralized Optimization

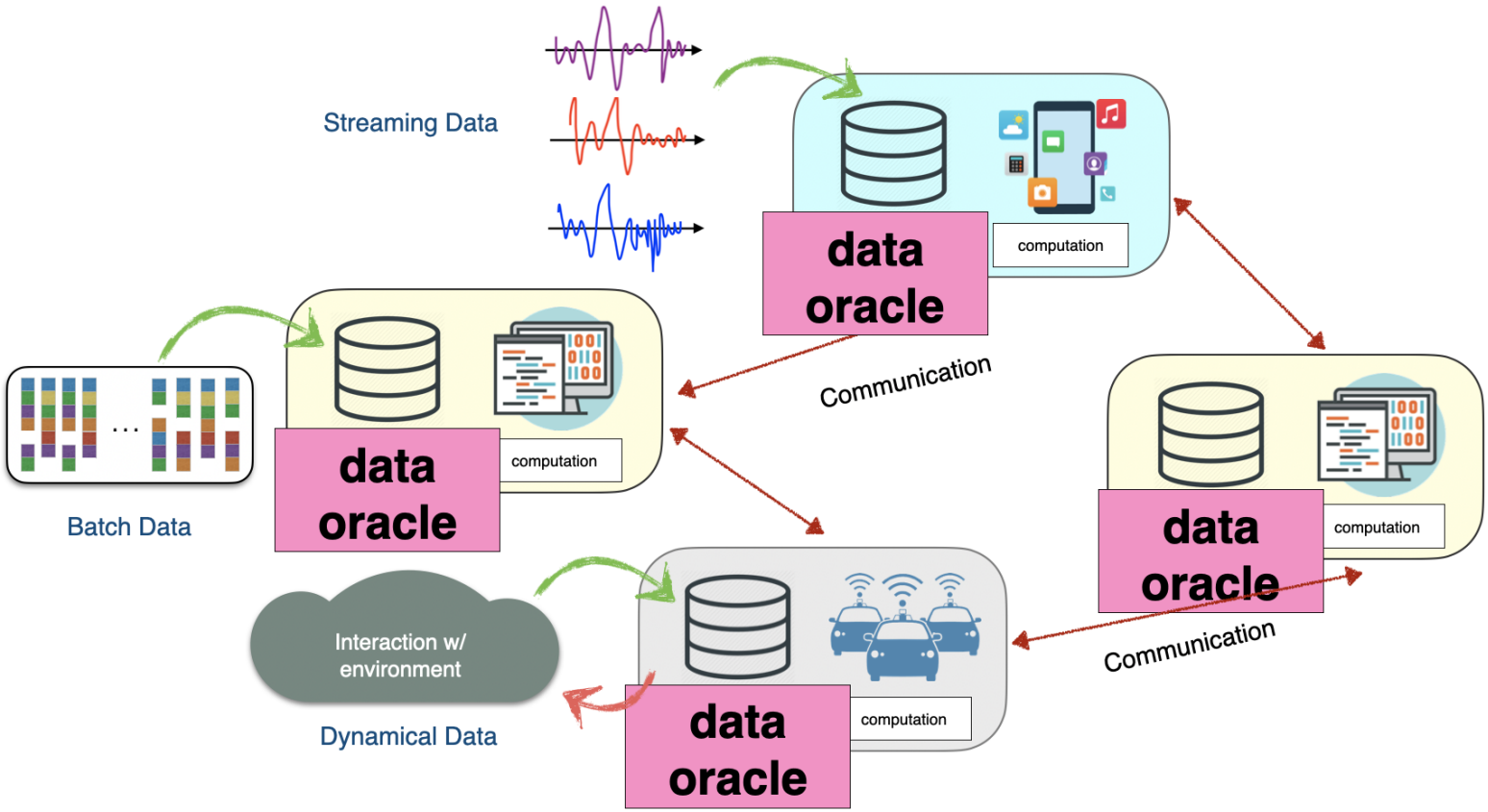
Some Key Elements

Figure 0.2: Key Elements



Some Key Elements

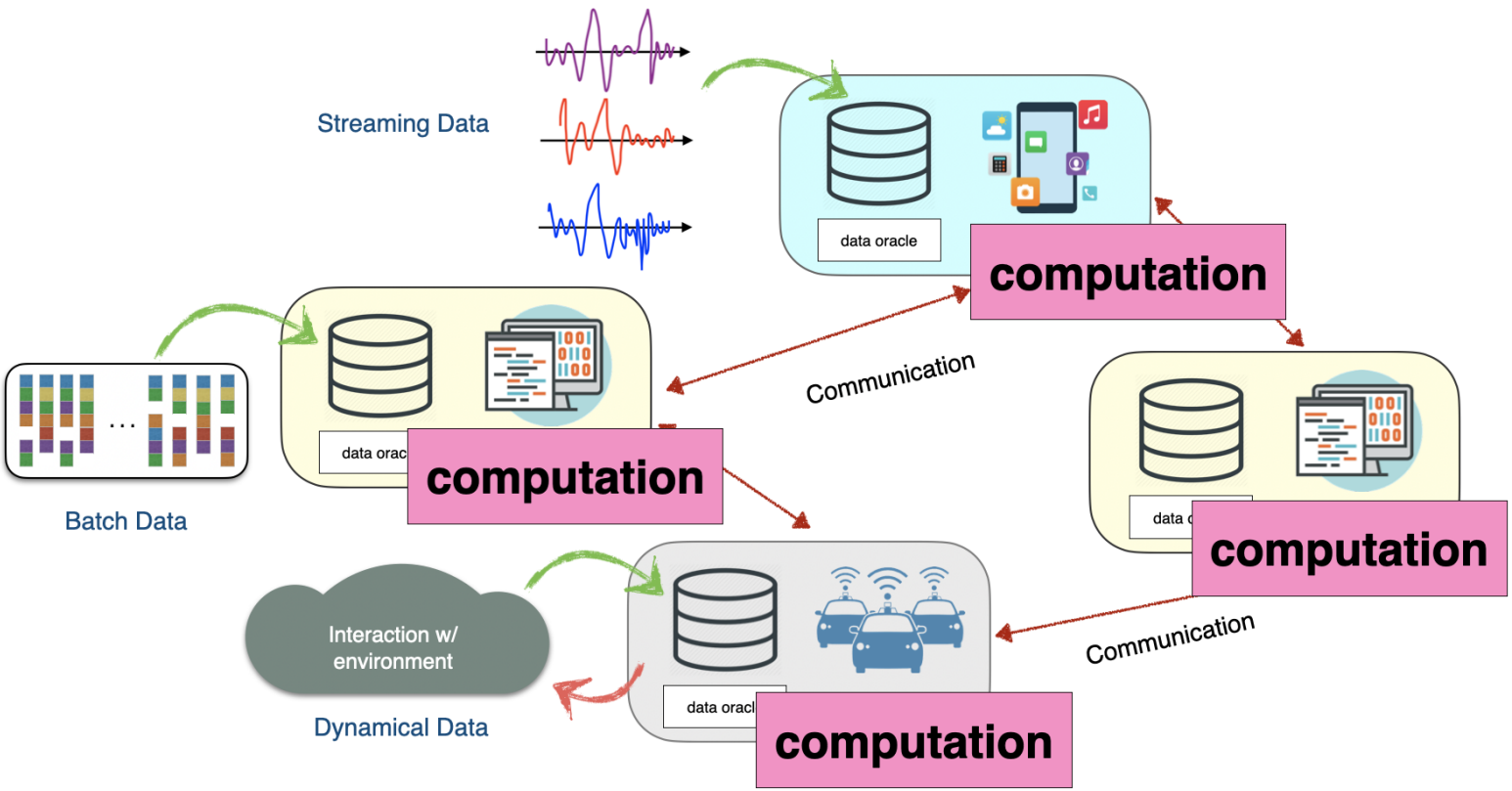
Figure 0.3: Key Elements



Data Oracle: Describing the data acquisition process.

Some Key Elements

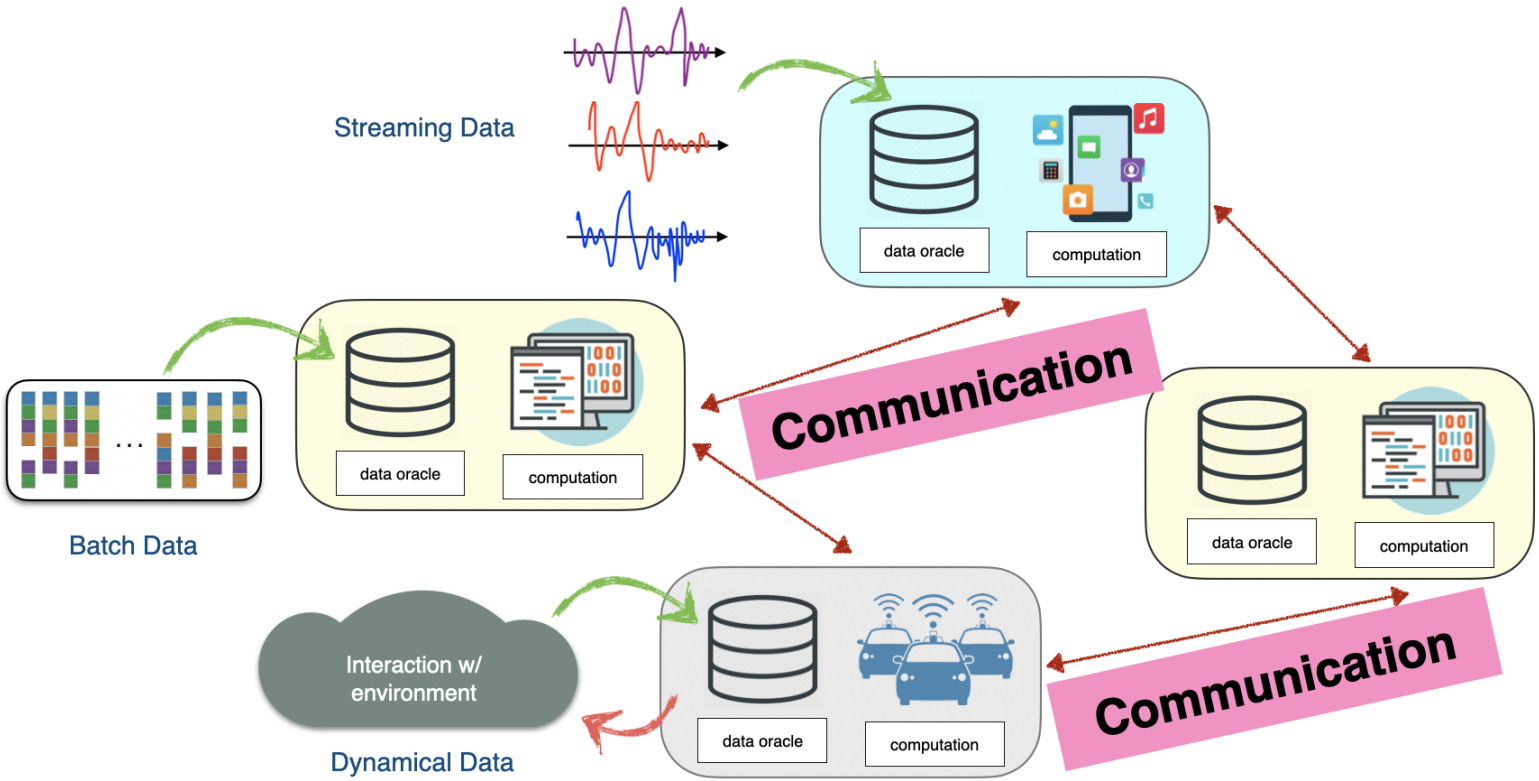
Figure 0.4: Key Elements



Computation Models: How the data is processed, which local problems to solved, etc...

Some Key Elements

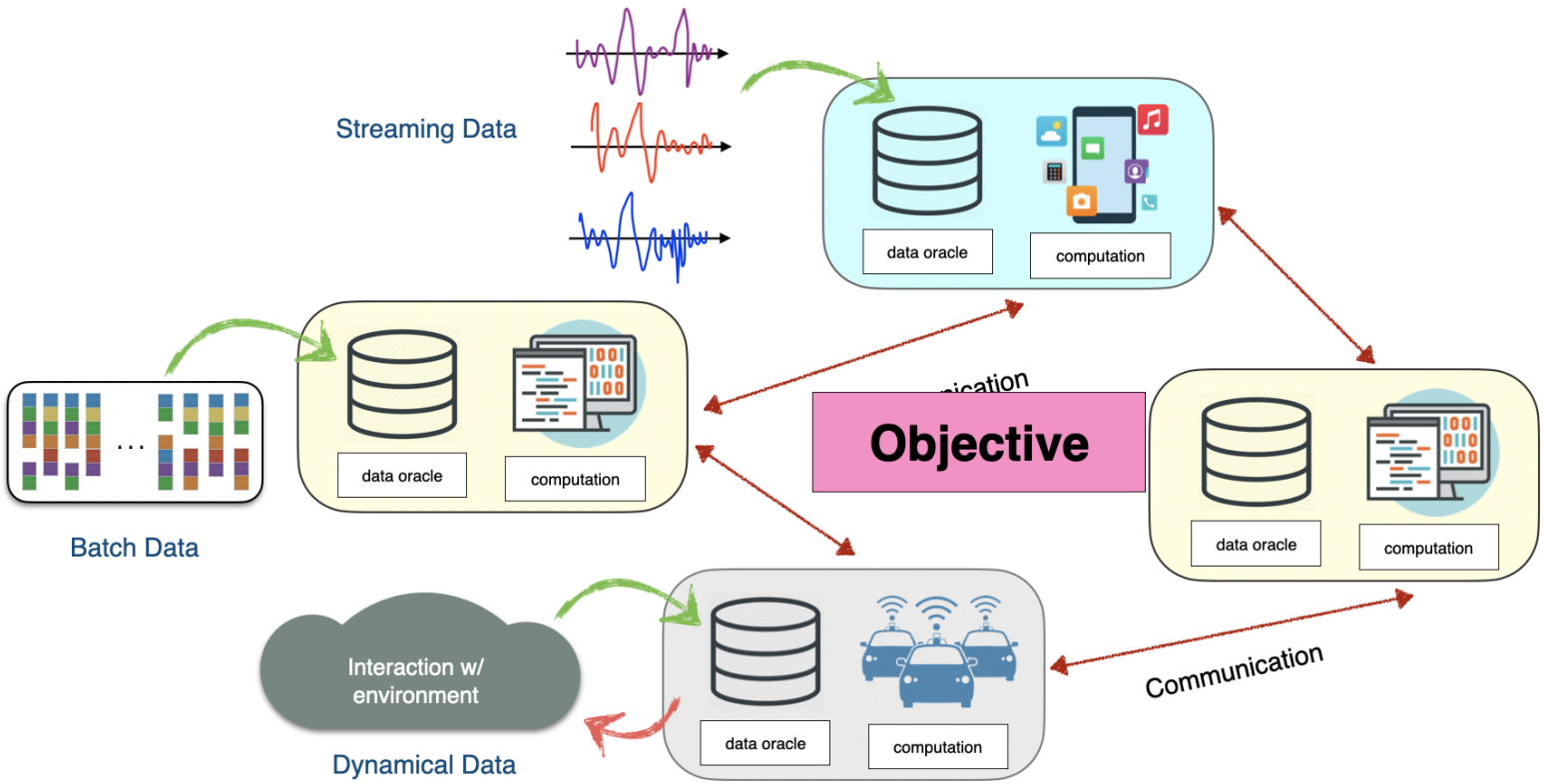
Figure 0.5: Key Elements



Communication: What information are exchanged, and how?

Some Key Elements

Figure 0.6: Key Elements



Objective: What kind of overall goal to achieve?

Key Research Question

- How to provide mathematical models for different applications?
- How to characterize and model different components of decentralized optimization?
- How to design algorithms, and provide analysis
- ...

Application: Decentralized Neural Network Training

- Distributed machine learning tasks [Balcan et al 15]
- Collaborative training deep neural networks, based on nodes' local data [Jiang et al 17] [Lian et al 17]
- High-dimensional problem, difficult and non-convex objectives

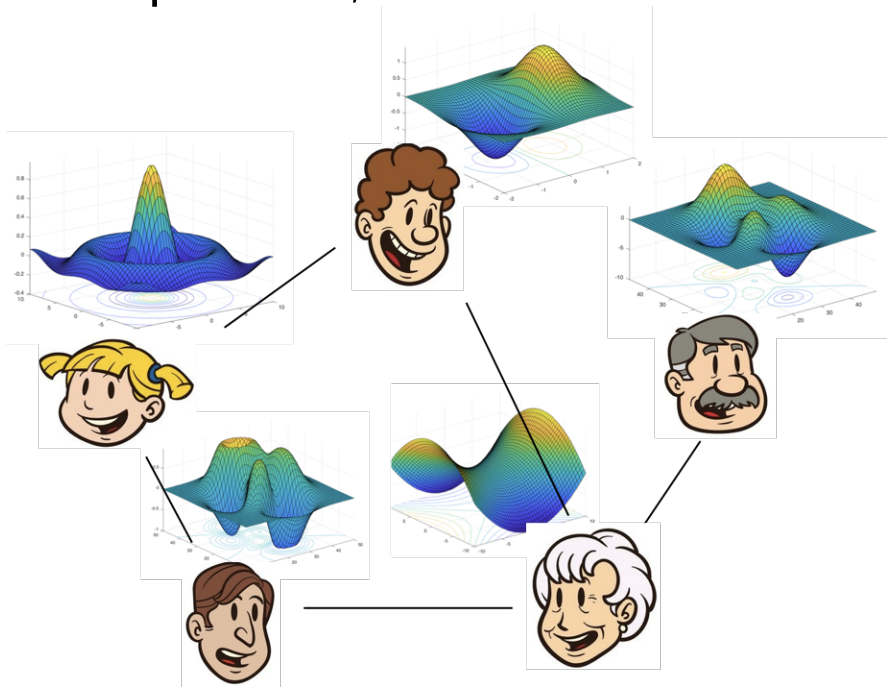
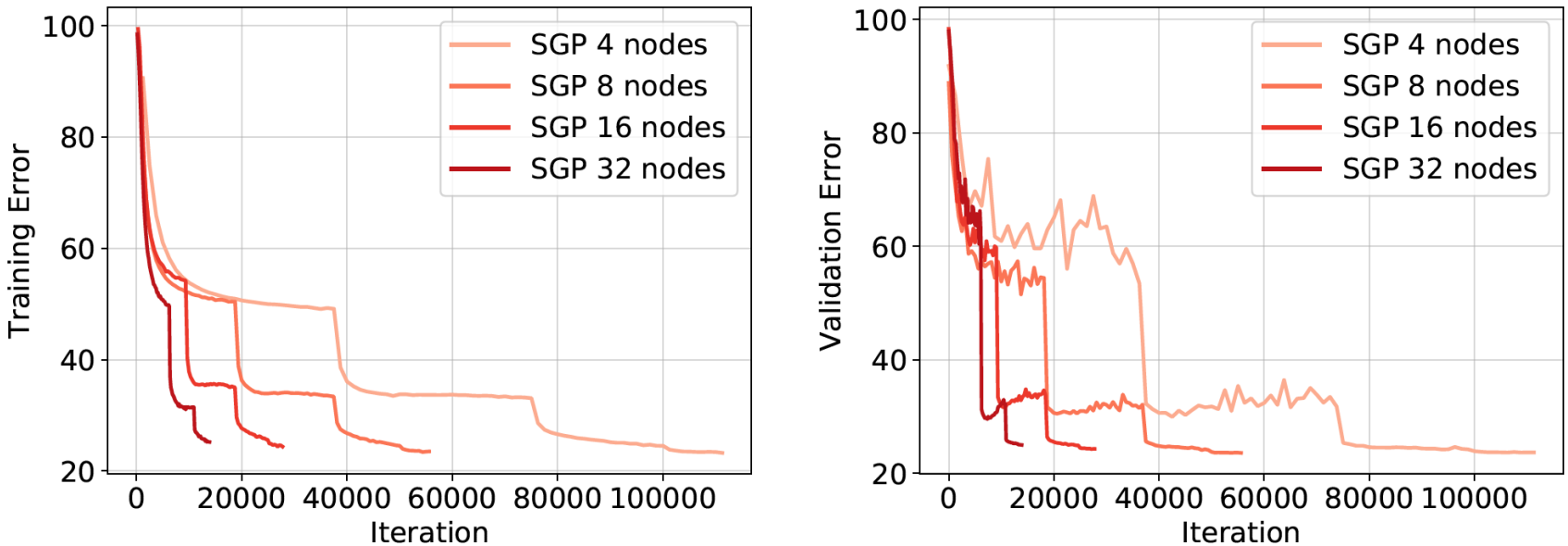


Figure 0.7: A distributed learning setting with distributed data.

Application: Decentralized Neural Network Training

- Even data are centrally located, distributed training using multiple machines/GPUs can significantly outperform centralized training [Assran et al 19]

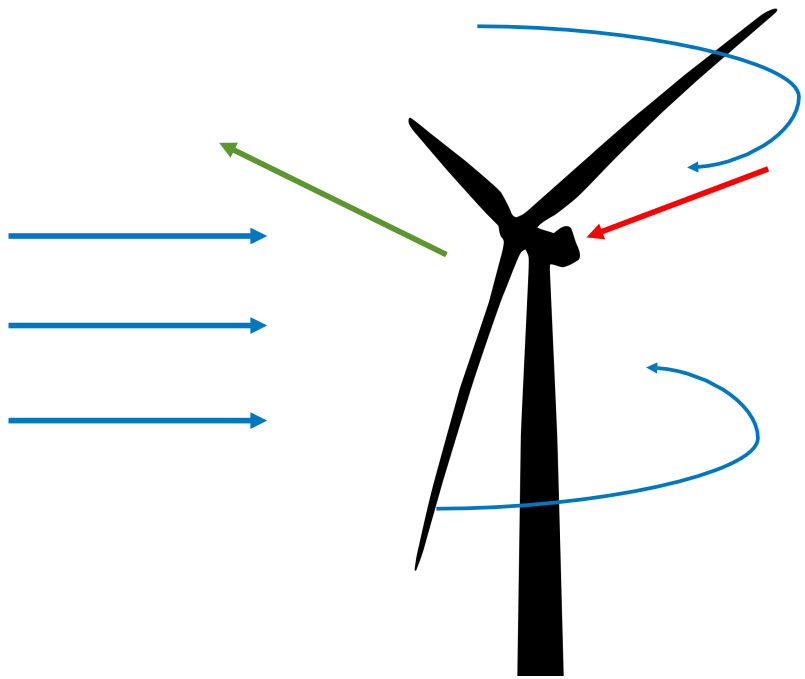


(a) Train (b) Validation

Figure 0.8: Distributed Training Improves Training/Testing Accuracy.

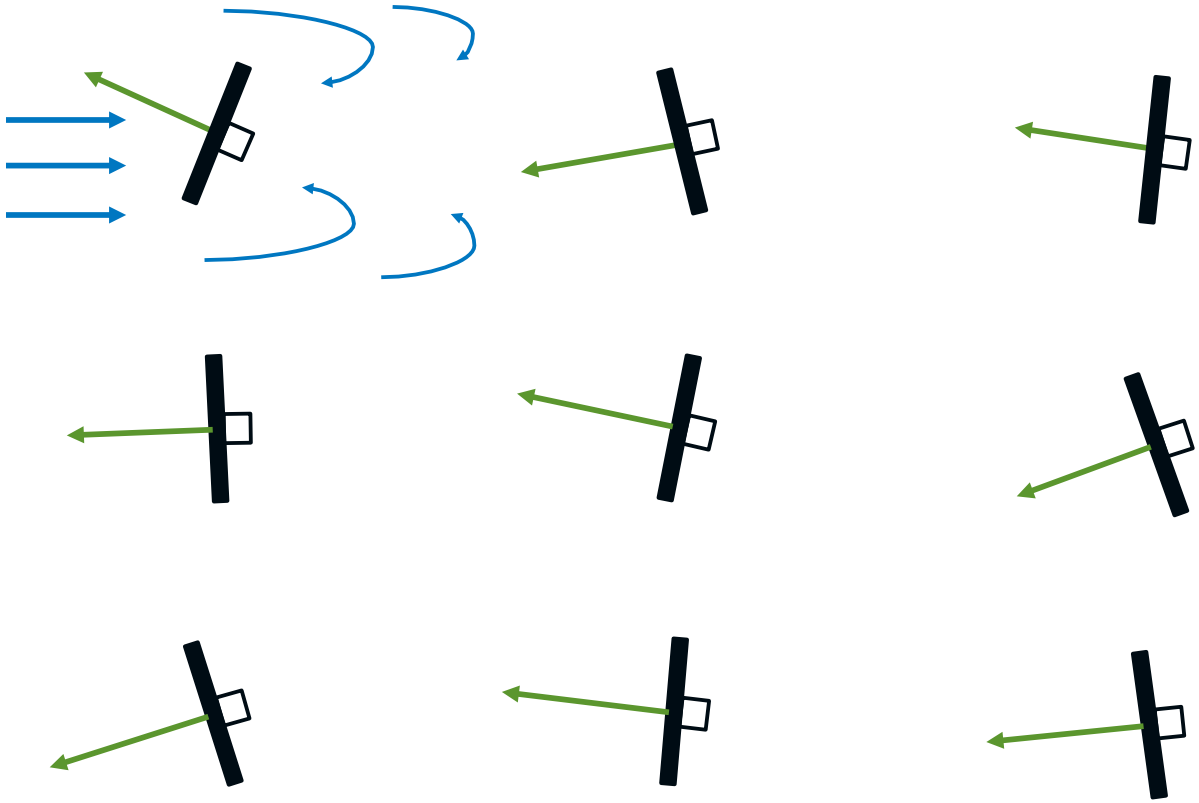
Application: Signal Processing/Power Systems

- Consider a wind turbine optimization problem (e.g., yaw angle)
- Traditionally optimized individually



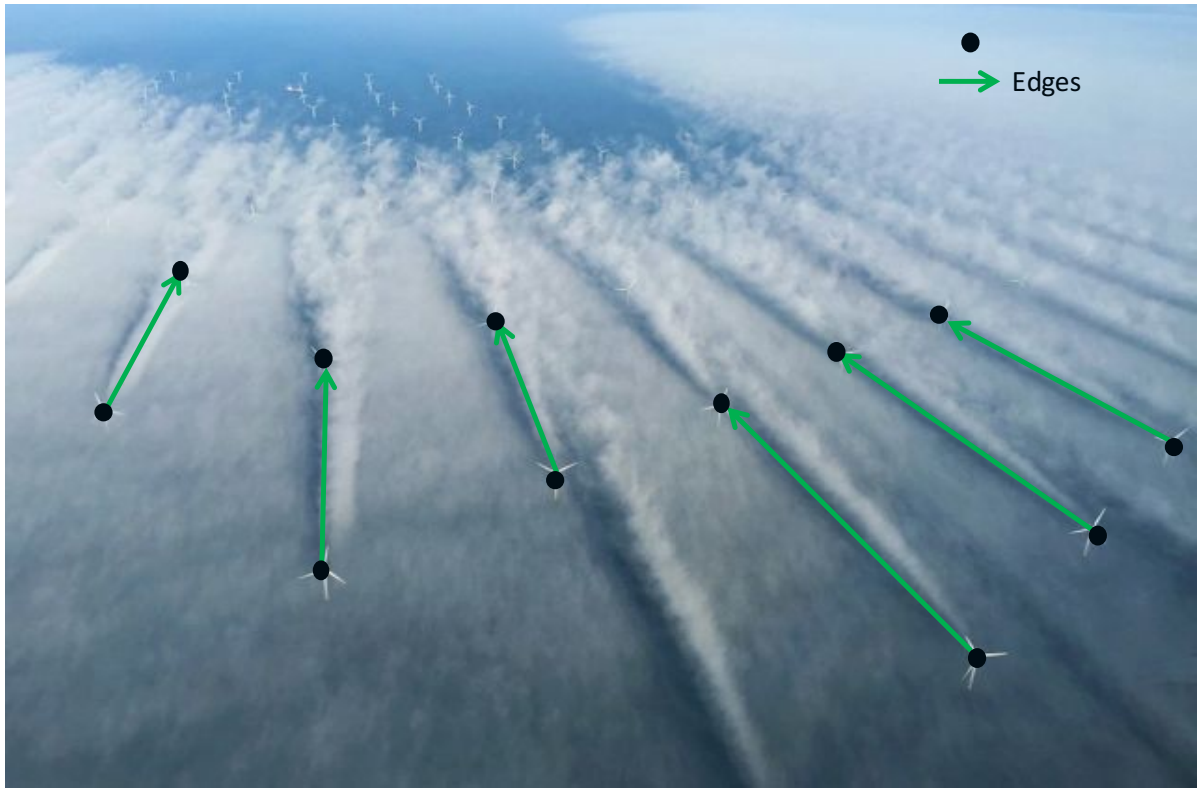
Application: Signal Processing/Power Systems

- But in typical setting, many wind turbine are collocated together



Application: Signal Processing/Power Systems

- But in typical setting, many wind turbine are collocated together



Application: Signal Processing/Power Systems

- Individual optimization can be highly suboptimal

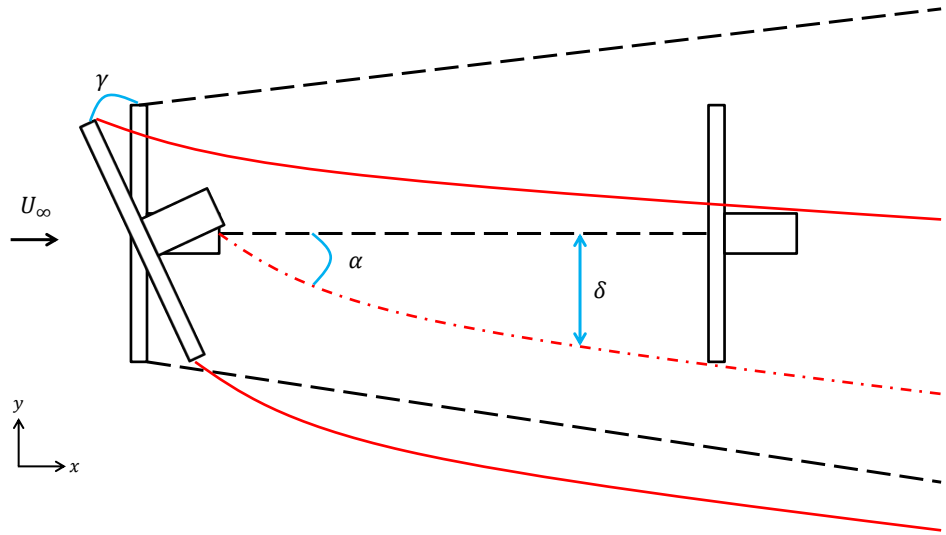


Figure 0.9: Two-turbine example of wake steering control

- No coordination between the up- and down stream turbines can cause the following issues:

  1. Lost power - yaw misalignment
  2. Constantly yawing - noisy wind vane signal

Application: Signal Processing/Power Systems

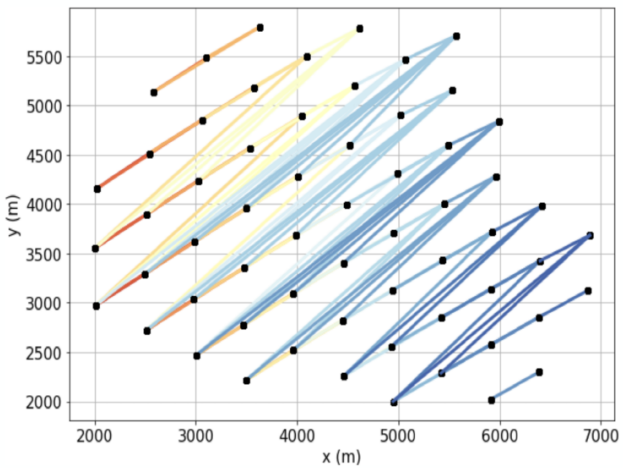
- **Solution**: Make the turbines connected
- Increased communication and coordination
  1. Consensus between turbines on direction
  2. Aligned with wind: increase power capture
  3. Robust wind direction measurement
  4. ....

Application: Signal Processing/Power Systems

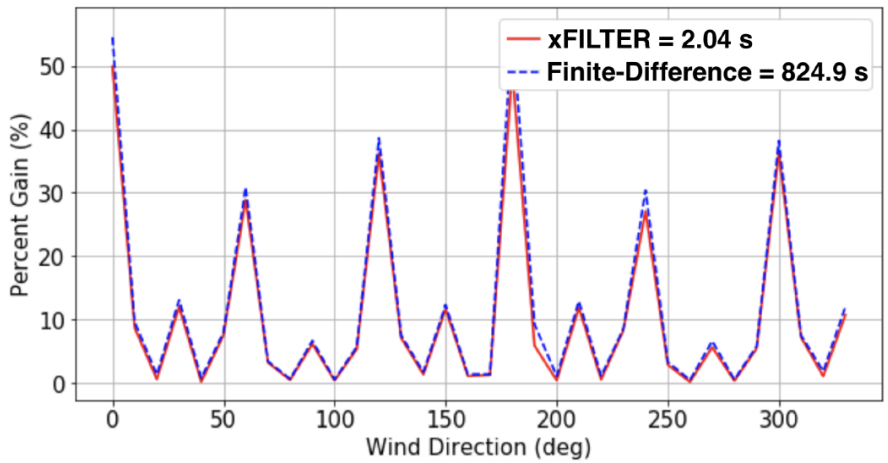
- **Challenge**: Individual optimization problem is complex
- The dynamics and power output of each turbine is a complicated function of parameters (such as yaw angles, wind speed, etc)
- Input/output relationships highly non-trivial (non-convex); they are usually simulated
    1. NREL's wind power simulation and modeling software (`https://www.nrel.gov/wind/data-tools.html`)

Application: Signal Processing/Power Systems

- Decentralized non-convex optimization significantly improves the practical performance [Annoni et al. 19]
- Simulated 60 NREL's 5MW turbines [Jonkman et al, 2009]



Graph structure of wind farm



100X faster than centralized method

Example 6: Decentralized Optimization

- Going back to decentralized optimization, it is clearly an important topic, with applications arising in different domains
- We will not go into detail about the formulation today, but we will discuss formulations and algorithms during our class
- The bottom line is, this is a special constrained optimization problem, and we can design specialized algorithms for it